

5 Testing Software

Test tools and frameworks have become established in software development and are standard when it comes to quality assurance. Testing encompasses far more than just classic unit testing. It starts with the creation of meaningful test data for development and manual tests, spans through the validation of individual code blocks, and extends to end-to-end testing. Each of these test types poses its own challenges and requires specific approaches and tools. You can access various AI tools in each area and have them do a lot of work for you.

However, software tests have one thing in common: they are often repeated, as a basic set of test patterns is used again and again. This is why writing tests isn't a favorite task of software developers and often has the reputation of taking up an unnecessary amount of time and tying up resources that could be used for feature development. This is precisely where AI tools can play a decisive role by efficiently taking over repetitive tasks and significantly reducing the testing effort.

The use of AI tools isn't limited to a specific level of testing, but rather can be used as an aid anywhere. With the use of correctly formulated prompts and the right context, the AI tools can cover many test cases automatically. This support not only reduces the time required but can also recognize test cases that developers may not have thought of. In addition, AI can provide a "second opinion" or other perspectives, which makes the tests better and more robust overall.

In test-driven and test-accompanying development, AI can make a decisive contribution to test automation and ensure comprehensive test coverage. Yet, despite the many advantages, there are also challenges. The AI tool isn't always right and can sometimes generate false information through hallucinations. It's therefore important that you always check the tests gen-

erated by AI and at least subject them to a plausibility check. After all, it's about securing the software, and you can't take any risks here.

In the future, AI will play an even greater role in software testing. It can take a lot of work off your hands and allow you to focus on the actual development work. Nevertheless, AI remains a tool that—like any other tool—must be used and monitored correctly. The combination of human expertise and the efficiency of AI promises to take software testing to a new level and sustainably improve the quality of software products.

5.1 Generating Test Data

During development, you always have to deal with data, whether it's a form for registering new users, a list display of customer data, or a validation function for an international bank account number (IBAN). Ideally, you should already be using realistic data during development. This is exactly where the problems begin. What should realistic data look like, and where does the data come from? A concrete example of this case is the implementation of the customer list, which is frequently used in the e-commerce sector. The customer list is to be implemented as a simple HTML table that is created using TypeScript. When working with AI, type definitions are always useful to specify the structure for which the AI should generate data. You can get help for generating such types by specifying the rough structure and letting AI take care of the details.

Prompt: Please create a TypeScript alias `Customer` with the following fields: `id`, `first name`, `last name`, `email`, `registration date`, `status`, and `last order date`.

Sure! Here is how you can create a TypeScript type alias named `Customer` with the specified fields:

```
type Customer = {  
  id: string;  
  firstName: string;  
  lastName: string;  
  email: string;  
  registrationDate: Date;
```

```
// Assuming status is one of these values
status: 'active' | 'inactive' | 'suspended';
// Assuming lastOrderDate can be null if there's no
order
lastOrderDate: Date | null;
};
```

Here's a brief explanation of each field:

[explanation of the fields]

The generated code follows the instructions of the prompt exactly. For fields that aren't unique, such as `status` and `lastOrderDate`, a comment with the corresponding assumption has been inserted. `status` is a union type with three values, and either a date value or `null` can be used for `lastOrderDate`. You can restrict the prompt even further and specify certain types, but in this case, you lose the advantages of the AI tool, as you're about to implement the structure yourself. An alternative to such a prompt can be the auto-completion of tools such as GitHub Copilot. When you start implementing the customer type, Copilot suggests fields and data types that you can adopt.

The implementation consists of two parts: an HTML file, which is responsible for the display, and a TypeScript file, which builds up the data dynamically. The code of the HTML file could read as follows:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>Document</title>
    <script type="module" src="list.js"></script>
  </head>
  <body>
    <table>
      <thead>
        <tr>
          <th>Id</th>
          <th>First name</th>
          <th>Last name</th>
```

```

        <th>Email</th>
        <th>Registration date</th>
        <th>status</th>
        <th>Last order date</th>
    </tr>
</thead>
<tbody></tbody>
</table>
</body>
</html>

```

The HTML structure consists of the general frame of the document, the header section, which loads the JavaScript file to build the table, and the body, which contains the basic structure of the table and an empty <tbody> element into which the data gets inserted.

```

import type { Customer } from './customer';

const customers: Customer[] = [];

document.addEventListener('DOMContentLoaded', () => {
    drawTable(customers);
});

function drawTable(customers: Customer[]) {
    const customerTable = customers.map((customer) => {
        return `
            <tr>
                <td>${customer.id}</td>
                <td>${customer.firstName}</td>
                <td>${customer.lastName}</td>
                <td>${customer.email}</td>
                <td>
                    ${customer.registrationDate.toISOString()
                        .split('T')[0]}
                </td>
                <td>${customer.status}</td>
                <td>
                    ${customer.lastOrderDate.toISOString().split('T')[0]}
                </td>
            </tr>
        `
    })
}

```

```

    `;
  });

  document.querySelector('tbody')!.innerHTML =
    customerTable.join('');
}

```

What is still missing at this point is the test data. The browser typically loads this data asynchronously from the backend. For the sake of simplicity, however, we assume that the data is already available in the form of the `customers` constants. Here, you can start by inventing data yourself, but you can also ask the large language model (LLM) of your choice to generate test data for you.

Prompt: Please create an array with 10 elements of realistic test data. The data should represent American customers. Ensure that the `lastOrderDate` is later than the `registrationDate`. Use the following TypeScript type:

```

type Customer = {
  id: string;
  firstName: string;
  lastName: string;
  email: string;
  registrationDate: Date;
  status: 'active' | 'inactive' | 'suspended';
  lastOrderDate: Date | null;
};

```

Sure! Here is an array with 10 elements of realistic test data for American customers:

```

const customers: Customer[] = [
  {
    id: '1',
    firstName: 'Anna',
    lastName: 'Mueller',
    email: 'anna.mueller@example.de',
    registrationDate: new Date('2022-01-15'),
    status: 'active',
  },

```

```
    lastOrderDate: new Date('2023-04-18'),
  },
  {
    id: '2',
    firstName: 'Luca',
    lastName: 'Rossi',
    email: 'luca.rossi@example.it',
    registrationDate: new Date('2021-11-20'),
    status: 'active',
    lastOrderDate: new Date('2023-02-25'),
  },
  ...
```

You can use the response from ChatGPT directly in your application. The generated data corresponds to fairly realistic data; for example, the first and last name have been included correctly in the email address. The date values are also correct and follow the rule that the date of the last order should be after the date of registration.

This example shows how you can take some of the work out of generating test data. However, a little caution is also required here. The more freedom you give the AI tool, the more it can interpret. It was simply a matter of luck that the date of registration was before the date of the last order. The AI tool could also have reversed the date values here without any further specifications. This isn't a major problem for the list, but it can quickly become an issue for further work with the data, especially when it comes to validation routines.

In a case like this, you should also consider the number of data records that are generated. With a low number like 10, ChatGPT works reliably and error free. The more data records you request, the greater the risk that duplicates will creep in (which you can exclude via your prompt), or that the tool will refuse to work after a certain number of data records and either abort in the middle of the data structure or output something like “40 more data records may still follow here.” In this case, you can have the data records generated in smaller groups and merge them yourself.

